



A college under Mapúa Malayan Colleges Laguna

Expanded MotorPH Payroll System Software Requirement Specification (SRS)

Prepared and Presented by:

Group 8

Bernardo, Michael Angelo

Grengia, Jafphet

Herrera, Christenne Jsele

Nocum, Ralph R-Nold

BSIT Major in Network and Cybersecurity

BSIT Major in Software Development

Term 3 AY 2024-2025

TABLE OF CONTENTS

Section

- 1 [Introduction](#)
 [Purpose](#)
 [Project Scope and Exclusions](#)
- 2 [Overall Description](#)
 [Product Perspective](#)
 [User Needs](#)
 [Assumptions & Dependencies](#)
- 3 [System Features & Requirements](#)
 [System Features](#)
 [Functional Requirements](#)
 [Non-Functional Requirements](#)
- 4 [Appendix](#)
 [Glossary](#)
 [Diagrams](#)

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document is revised and updated to precisely capture the evolving requirements, design choices, and implementation strategy for the MotorPH payroll system expansion. The primary driving force for this revision stems from the necessity to address changing business needs while diligently preserving the system's fundamental functionality and data integrity. The expansion is specifically directed towards integrating more advanced SQL queries, enhancing reporting functionalities, and improving employee management features, thereby complementing our already robust database-integrated solution. This SRS serves as a foundational document, guiding the development team and providing a clear understanding of the expanded system's capabilities for all stakeholders.

This report is meticulously organized to offer extensive coverage of the extended payroll system's requirements. It begins by outlining the purpose and scope of the project, followed by a detailed product perspective that highlights key features and functionalities. The document then delves into the specific user needs addressed by the expansion, along with crucial assumptions and dependencies. Subsequent sections will include updated design diagrams reflecting the new MVVM architectural style and details on modified sections of the codebase (e.g., `Employee.java`, `EmployeeLoader.java`, `PayrollAdjustment.java`), as well as newly introduced components (e.g., `ReactivationManager.java`, `ExportManager.java`). The report will comprehensively address functional requirements, non-functional specifications, system constraints, and implementation considerations for the extended payroll system. A comprehensive glossary of terms will also be provided for clarity.

This SRS is intended for a diverse group of stakeholders who will interact with or be impacted by the MotorPH payroll system expansion. These include:

- **Project Managers:** To ensure the project aligns with strategic business objectives and is delivered within scope, budget, and timeline.
- **Development Team:** To understand the detailed technical requirements and implement the system accurately.

- **Quality Assurance (QA) Team:** To develop test plans and ensure the system meets all specified requirements and quality standards.
- **HR Managers:** To understand how the system will streamline employee lifecycle management, reporting, and compliance.
- **Department Heads:** To utilize enhanced reporting and employee status monitoring for workforce planning.
- **Payroll Personnel:** To leverage improved payroll processing workflows, automated calculations, and error prevention features.
- **System Administrators:** To manage system performance, security, and maintenance, including data backup and recovery.
- **End-Users:** To understand the new functionalities and improved user experience.

1.2 Project Scope and Exclusions

Scope: The MotorPH payroll system, currently a robust database-integrated Java desktop application, is undergoing a significant expansion to address evolving business needs and enhance its overall capabilities. The updated scope of this software product is to transform the existing system into a more efficient, user-friendly, and comprehensive payroll management solution. This involves a strategic upgrade that retains the fundamental structure while integrating contemporary design patterns and sophisticated functionalities.

Key Benefits, Objectives, and Goals of the Expanded System:

- **Increased Operational Efficiency:** Streamline payroll processing through optimized SQL queries, automated calculations, and multi-format report generation, significantly reducing manual effort and processing time.
- **Enhanced Data Integrity and Management:** Implement logical deletion with status flags instead of physical record deletion, ensuring data continuity and historical record retention. This objective directly supports corporate compliance and audit trail requirements.
- **Improved Employee Lifecycle Management:** Provide comprehensive tools for managing employee status changes,

including efficient deactivation and reactivation procedures, aligning with HR department's need for accurate workforce tracking.

- **Superior Reporting and Analytics:** Offer dynamic, multi-format export capabilities for payslips and various reports (PDF, CSV, Excel), enabling better decision-making for department heads and HR managers. This directly contributes to corporate goals of data-driven insights.
- **Enhanced User Experience and Maintainability:** Adopt the MVVM (Model-View-ViewModel) architectural pattern to improve code organization, maintainability, and testability. This will lead to a more intuitive and less error-prone user interface, boosting user satisfaction and reducing support overhead.
- **Scalability and Adaptability:** Lay the groundwork for future system growth by implementing a modular design and robust database integration, supporting potential integrations with other corporate systems and accommodating future business requirements.

This expansion directly relates to MotorPH's corporate goals of **digital transformation**, **operational excellence**, and **employee satisfaction**. By automating and streamlining critical payroll functions, the system will contribute to more accurate financial reporting, reduce administrative burden, and ensure timely and correct compensation for employees, thereby fostering a more positive work environment.

Exclusions: The following elements or activities fall outside the defined boundaries of this phase of the MotorPH payroll system expansion:

- **Third-Party System Integration:** Direct, real-time integration with external HR information systems (HRIS), enterprise resource planning (ERP) systems, financial accounting packages (e.g., SAP, Oracle Financials), or government compliance platforms (e.g., tax authorities, social security agencies) is explicitly excluded from this current phase. Data exchange with such systems, if required, will initially be handled via manual export/import of reports.

- **Employee Performance Management:** The system does not extend to functionalities related to employee performance evaluations, goal setting, 360-degree feedback, performance review workflows, or other aspects of performance management. These activities are assumed to be covered by a separate or forthcoming performance management system.
- **Mobile Application Development:** The development of native mobile applications for employees (e.g., for viewing payslips, applying for leaves) or HR personnel is not part of this expansion. The system remains a desktop application for this iteration.
- **Comprehensive Web Interface:** While UI improvements are planned for the desktop application, a complete, browser-based web interface for the payroll system is not included in this scope. This is considered a potential future phase.
- **Advanced Predictive Analytics:** Features such as predictive analytics for budget forecasting, attrition prediction, or complex workforce planning based on advanced statistical models are excluded. The reporting enhancements focus on retrospective and current data analysis.
- **Multi-language Support (Internationalization/Localization):** The system will operate solely in English. Support for multiple languages or localization features for different geographical regions is not included.
- **Real-time Notification Services:** Automated real-time notifications via email, SMS, or in-app push notifications for payroll events (e.g., payslip availability, payroll processing completion) are not within the current scope.
- **Single Sign-On (SSO) Integration:** Authentication integration with external enterprise-level Single Sign-On (SSO) systems is not included. User authentication will continue to be managed within the payroll system's existing security framework.
- **Recruitment and Onboarding Management:** Features related to applicant tracking, resume management, interview scheduling, or the digital onboarding of new hires are excluded. The system focuses on employee data once they are onboarded.

2. Overall Description

2.1. Product Perspective

The MotorPH payroll system, enlarged from its original Java desktop application, represents a considerable upgrade to the existing system. While retaining its fundamental structure, the system incorporates contemporary design patterns and sophisticated functionality to meet evolving business needs. It will continue to operate as a standalone desktop application, integrating with a MySQL database.

Key Features and Functionalities:

■ Employee Management Enhancement:

- **Advanced Employee Record Management:** The system will allow for comprehensive management of employee records, including personal details, contact information, job titles, departments, and payroll-specific data.
 - **Illustrative Example:** An HR administrator can update an employee's marital status, change their department from "Marketing" to "Sales," or modify their base salary, with all changes logged for audit purposes.
- **Support for Logical Deletion:** Instead of permanently deleting employee records, the system will implement a "logical deletion" mechanism. This involves marking an employee's record as "inactive" using a status flag, preserving historical data while making the employee invisible in active listings.
 - **Illustrative Example:** When an employee resigns, an HR user can "deactivate" their record. The employee will no longer appear in active employee lists, but their past payroll data and other historical information will remain accessible for reporting and compliance.
- **Status-Driven Employee Tracking (Active/Inactive):** Enhanced search and filtering capabilities will allow users to easily differentiate between active and inactive employees.
 - **Illustrative Example:** A payroll officer can quickly generate a list of "active" employees for the current

payroll run, or view "inactive" employees to find historical payroll data.

- **Employee Reactivation Process:** A dedicated process will be available to reactivate previously deactivated employees, streamlining the re-hiring process.
 - **Illustrative Example:** If a former employee is rehired, an HR user can use the "Reactivation Module" to change their status from "inactive" back to "active," updating necessary details without creating a new record.
- **Enhanced Search Functionality:** Sophisticated search features will allow filtering by various criteria, including employee status.
 - **Illustrative Example:** A manager can search for employees by name, employee ID, department, or by their active/inactive status, allowing for quick data retrieval.

■ **Payroll Processing Improvements:**

- **Efficient Algorithms for Salary Calculation:** Optimized computational techniques will be implemented for accurate and faster calculation of salaries, including regular pay, overtime, bonuses, deductions, and statutory contributions.
 - **Illustrative Example:** The system will accurately calculate an employee's gross and net pay based on their hours worked, applied leave, and predefined deductions, ensuring compliance with company policies and labor laws.
- **Robust Transaction Management:** Implementation of transaction management to maintain data integrity during critical payroll processing operations, ensuring that either all changes related to a payroll run are committed or none are.
 - **Illustrative Example:** During a payroll run affecting multiple employees, if a system error occurs mid-process, transaction management will ensure that no partial updates are saved to the database, preventing data corruption.

- **Automated Paystub Generation:** The system will automate the generation of individual paystubs with support for multiple export formats.
 - **Illustrative Example:** After a payroll run, the system can automatically generate PDF paystubs for all employees, which can then be easily distributed.
- **Improved Error Handling and Data Validation:** Comprehensive data validation rules will be applied at input points, and robust error-handling mechanisms will provide clear feedback to users, minimizing manual processing errors.
 - **Illustrative Example:** When entering employee hours, the system will prevent negative values or alert the user to unusually high entries, guiding them to correct potential data entry mistakes.

■ **Reporting and Analytics:**

- **Dynamic Report Generation System:** The system will offer a flexible framework for generating various payroll and employee-related reports.
 - **Illustrative Example:** An HR manager can generate a "Departmental Salary Report" for a specific month, showing total salaries, deductions, and net pay per department.
- **Support for Diverse Output Formats:** Reports and paystubs can be exported in multiple formats, including PDF, CSV, and future compatibility with Excel.
 - **Illustrative Example:** A payroll officer can export a "Summary Payroll Report" in CSV format for further analysis in a spreadsheet program, or as a PDF for easy printing and distribution.
- **Parameterized Queries for User-Defined Reporting:** Users will be able to define specific criteria for their reports using parameterized queries, allowing for highly customized reporting.
 - **Illustrative Example:** A department head can generate a report of all "active" employees in their department who have accrued more than 10 vacation leave days.

- **Data Aggregation Functionality:** The system will provide capabilities for aggregating data to generate summary reports and managerial dashboards.
 - **Illust illustrative Example:** The system can aggregate total gross pay, total deductions, and total net pay across the entire company for a fiscal quarter, presenting this data in a concise summary for management.

■ **Technical Infrastructure:**

- **MVVM Pattern Application:** The codebase will be reorganized using the Model-View-ViewModel (MVVM) architectural pattern to enhance maintainability, testability, and separation of concerns.
- **JDBC Database Connection with Connection Pooling:** Java Database Connectivity (JDBC) will be used for direct database interaction, complemented by connection pooling (e.g., Apache Commons DBCP) to optimize performance and resource utilization.
- **Detailed Audit Logging:** Comprehensive logging mechanisms will record significant system activities, including data modifications, user logins, and critical payroll runs, ensuring compliance and aiding in troubleshooting.
- **Modular Implementation:** The system will be designed with a modular approach to enable easier future feature enhancements and potential integrations.

2.2. User Needs

The expanded MotorPH payroll system aims to address critical user needs and pain points identified through stakeholder consultations and an analysis of current operational challenges. These needs are prioritized based on their impact on efficiency, compliance, and overall user experience. We acknowledge that these needs may evolve, and the system design maintains flexibility for future refinements.

User Role	Key User Needs	Pain Points Addressed by Expansion
HR Managers	- Effective Employee Lifecycle Management: Need streamlined processes for managing employee hiring, status updates (active/inactive), and efficient reactivation procedures for returning employees. - Detailed Reporting & Compliance: Require comprehensive and customizable reports for payroll analysis, compliance audits, and internal decision-making. - Data Export & Integration: Need the ability to easily export data for various purposes, including external system integration and regulatory reporting. - Audit Trail & Historical Records: Demand a robust audit trail to track all changes to employee records and payroll transactions, ensuring accountability and compliance.	- Manual Data Entry & Redundancy: Reduces manual efforts in updating employee statuses and re-entering data for returning employees. - Limited Reporting Flexibility: Overcomes the inflexibility of current reporting, allowing for more specific data extraction and analysis. - Time-Consuming Data Preparation: Streamlines the process of preparing data for external systems or compliance reports, saving significant time. - Lack of Data Traceability: Provides clear visibility into who changed what and when, improving accountability and simplifying audits.
Department Heads	- Employee Status Monitoring: Need quick and accurate visibility into the active/inactive status of employees within their department for workforce planning and management. - Departmental Payroll Reporting: Require secure access to relevant payroll reports specific to their department, with appropriate security controls. - Rapid Data Retrieval: Need fast and efficient search and filtering capabilities to locate individual employee data within their teams. - Budget & Resource Allocation Insights: Require summary reporting to aid in budget planning and	- Difficulty Tracking Employee Status: Simplifies tracking of current team members, especially in dynamic environments with frequent changes. - Limited Access to Relevant Data: Provides self-service access to departmental payroll overviews, reducing reliance on HR for basic inquiries. - Slow Information Access: Enables quick lookups of employee details without sifting through extensive records. - Inaccurate or Delayed Budget Data: Offers more immediate and accurate data for financial planning within the department.

	resource allocation.	
Payroll Personnel	- Improved Payroll Processing Workflow: Need a more automated, efficient, and error-preventing workflow for calculating and processing payroll. - Error Prevention & Validation: Demand features that minimize manual processing errors and highlight discrepancies before finalization. - Batch Processing Efficiency: Require the ability to efficiently process multiple employee records concurrently. - Integration with Financial Systems: Need seamless integration capabilities with current financial and accounting systems (even if currently via export/import) to avoid reconciliation issues.	- Manual Calculation Errors: Significantly reduces the incidence of calculation mistakes and re-work. - Time-Consuming Error Correction: Identifies and flags potential errors earlier in the process, reducing time spent on reconciliation post-processing. - Inefficient Handling of Large Datasets: Accelerates the processing of payroll for many employees simultaneously. - Data Silos & Reconciliation Issues: Eases the transfer of payroll data to accounting systems, minimizing discrepancies and manual reconciliation efforts.
System Administrators	- Comprehensive Data Backup & Recovery: Need robust tools and procedures for ensuring business continuity through reliable data backup and rapid recovery functions. - System Performance Monitoring: Require tools to monitor system performance for optimization and troubleshooting. - Security & Access Control: Demand robust user access control and permission management to ensure security compliance. - System Maintenance & Troubleshooting: Need effective capabilities for routine system maintenance (e.g., database optimization) and efficient error logging for debugging.	- Risk of Data Loss & Downtime: Mitigates the risk of critical payroll data loss and minimizes system downtime in case of incidents. - Lack of Performance Insight: Provides visibility into system bottlenecks, allowing for proactive performance improvements. - Vulnerable Security Posture: Ensures only authorized personnel access sensitive payroll data, enhancing data security. - Difficult Debugging & Maintenance: Simplifies identifying and resolving system issues, reducing maintenance overhead.

■ Flexibility for Refinement:

The prioritization and categorization above serve as a high-level guide. As the development progresses and further feedback is gathered from user acceptance testing (UAT) and iterative demonstrations, these needs will be refined. The modular design, MVVM architecture, and direct JDBC control are intentionally chosen to provide the flexibility required to adapt to evolving user needs and incorporate new functionalities efficiently.

2.3. Assumptions and Dependencies

This section outlines assumed factors that may impact the requirements outlined in this SRS, with a particular focus on user-centric aspects. It also lists external dependencies that could influence the project's success.

■ Assumptions

Assumptions are factors believed to be true, but which, if proven false, could impact the project's requirements, scope, or timeline.

● User-Centric Assumptions:

- **Basic Computer Literacy:** It is assumed that all end-users (HR Managers, Department Heads, Payroll Personnel, System Administrators) possess fundamental computer literacy skills, including familiarity with desktop applications, basic navigation, and data entry.
- **Familiarity with Payroll Processes:** HR personnel and payroll staff are assumed to have a comprehensive understanding of MotorPH's existing payroll computation rules, deductions, benefits, and general payroll processing workflows. The system expansion will automate these processes, not define them from scratch.

- **Adherence to Data Entry Procedures:** Users responsible for data entry (e.g., HR personnel updating employee records) are assumed to adhere to established data entry protocols, including accuracy and completeness of information, and to utilize the system's validation features.
 - **Willingness to Adapt to New Workflows:** Users are assumed to be receptive to adopting the new functionalities and revised workflows introduced by the expanded system, such as logical deletion and the dedicated reactivation module, following appropriate training.
 - **Availability for Training:** Key users across all roles are assumed to be available and willing to participate in training sessions provided for the expanded system to ensure effective utilization of new features.
 - **Management Support for Policy Changes:** Any minor policy or procedural adjustments necessitated by the system's enhanced features (e.g., formalizing the logical deletion process) are assumed to have full management support and communication.
- **Technical Assumptions:**
 - **Existing Hardware Infrastructure:** The current hardware setup at MotorPH (servers, workstations) is assumed to be capable of sustaining the increased system demands, particularly regarding database performance and application responsiveness, without significant upgrades.
 - **MySQL Database Version Compatibility:** The current version of the MySQL database in use (MySQL Workbench 8.0 is mentioned for management) is assumed to fully support all planned optimization features, triggers, stored procedures, and data types required for the expansion.
 - **JDK 25 Stability and Support:** The chosen Java Development Kit (JDK 25) is assumed to be stable, fully supported, and compatible with all planned libraries and

frameworks throughout the project's development and initial deployment phases.

- **Stable Network and Database Connectivity:** It is assumed that the network infrastructure and database connectivity within MotorPH's environment will remain stable and reliable during system operation, minimizing performance bottlenecks or data access issues.

- **Business Process Assumptions:**

- **Stable Payroll Business Logic:** The core payroll computation rules and business logic (e.g., overtime calculation rates, deduction types, tax bracket application) are assumed to remain consistent during the project's development phase. Any changes to these rules will necessitate re-evaluation of requirements.
- **Defined Employee Status Change Procedures:** MotorPH is assumed to have clearly defined organizational procedures for employee status changes (hiring, resignation, deactivation, re-hire), which the system's logical deletion and reactivation features will then automate.
- **Compliance with Data Retention Policies:** MotorPH is assumed to have established data retention policies that align with legal and regulatory frameworks, which the system's historical record preservation capabilities will support.
- **Existing Backup and Disaster Recovery:** Pre-existing backup and disaster recovery procedures for the current payroll database are assumed to be properly defined and executed, and the expanded system will integrate with these.

■ Dependencies

Dependencies are external factors or entities that the project relies upon for its success, but which are not directly controlled by the project team.

- **External System Dependencies:**

- **MySQL Database Server:** The project is critically dependent on the continuous availability, performance, and stability of the MotorPH MySQL database server for all data storage, retrieval, and processing operations.
- **NetBeans IDE 25:** The development team relies on the NetBeans IDE 25 for ongoing development, debugging, and maintenance work. Availability of licenses and support for this IDE is crucial.
- **Apache Commons DBCP:** The successful implementation of database connection pooling for performance optimization is dependent on the stable and compatible integration of the Apache Commons DBCP library.
- **JUnit Framework:** The project's quality assurance process relies heavily on the JUnit framework for unit testing. Its continued functionality and compatibility are essential for verifying code integrity.
- **Operating System Environment:** The proper functioning of the desktop application is dependent on the target operating system (e.g., Windows, macOS, Linux) and its underlying infrastructure.

- **Organizational Dependencies:**

- **Stakeholder Availability:** The project is dependent on the timely availability of key stakeholders (HR, Payroll, IT Management) for requirements verification, feedback sessions, user acceptance testing (UAT), and final approvals. Delays in this could impact the project timeline.
- **IT Department Support:** Ongoing support from MotorPH's IT department is crucial for system deployment, database administration (e.g., schema modifications, backup verification), network configuration, and general technical infrastructure support.

- **Management Approval:** Approval from MotorPH management is required for significant feature implementations, system changes, resource allocation, and project phase transitions.
 - **Training Resource Allocation:** The successful adoption of the new system is dependent on the allocation of adequate resources for developing training materials and conducting end-user training sessions.
-
- **Technical Dependencies:**
 - **Database Schema Modification Approval:** Any proposed modifications to the existing database schema required for the new features (e.g., adding **status** fields, creating new tables) must be reviewed and approved by the database administration team. Coordination for implementation is also critical.
 - **Data Migration Plan and Execution:** The addition of new fields (like the **status** flag) to existing employee records will require a clear data migration plan and its successful execution, which is dependent on database expertise and potential downtime coordination.
 - **Performance Testing Environment:** The availability of a dedicated performance testing environment is crucial for conducting load and stress testing to ensure the expanded system meets non-functional performance requirements before production deployment.
 - **Security Policies and Guidelines:** Adherence to MotorPH's internal security policies and guidelines is a dependency for all development and deployment activities, particularly concerning data access, encryption, and user authentication.

3. System Features and Requirements

3.1. System Features

This section outlines the key functionalities and features of the expanded MotorPH payroll system, providing detailed descriptions for each. These features are designed based on the updated design diagrams and the overall expansion plan, ensuring a comprehensive and improved payroll management solution. Each feature is presented with its primary purpose and the specific functionalities it encompasses.

3.2. Functional Requirements

The expanded MotorPH payroll system is designed to provide a comprehensive solution for managing employee payroll and related HR functions. It will offer HR managers streamlined employee lifecycle management and detailed reporting, while department heads will gain better visibility into employee status and departmental payroll. Payroll personnel will benefit from automated, error-preventing workflows, efficient batch processing, and seamless integration with financial systems. System administrators will have robust tools for data backup, performance monitoring, security, and maintenance, ensuring an efficient, compliant, and user-friendly experience for all key stakeholders.

■ Employee Data Management

- **Purpose:** To serve as the central, secure repository for all employee information, enabling efficient lifecycle management from onboarding to potential deactivation, ensuring data accuracy for HR operations, payroll processing, and regulatory compliance.
- **Description:** This module handles the comprehensive creation, retrieval, updating, and reporting of detailed employee information. It encompasses personal demographics, employment specifics, financial foundational data, government identifiers, and manages their active/inactive status without permanent deletion.
- **Functionalities:**
 - **CreateNewEmployee(employeeData):** Captures and stores all initial demographic (name, address, birthday, phone), employment (designation, status, supervisor), and foundational

financial (basic salary, hourly rate) data for a new hire. This includes generating a unique **Employee ID (EID)** and setting initial login credentials.

- **GetEmployeeDetails(employeeId):** Retrieves a complete profile of a specific employee, including all personal, contact, employment, and financial base data, using their unique EID.
- **UpdateEmployeeDetails(employeeId, updatedData):** Allows authorized users to modify any aspect of an employee's record, such as changes to designation, address, contact number, or basic salary. This includes dedicated methods for individual fields like `testSetFirstName()`, `testSetLastName()`, `testSetAddress()`, etc.
- **UpdateEmployeeStatus(employeeId, newStatus):** Sets an employee's employment status (e.g., Active, On-Leave, Inactive), controlling their visibility in active lists and system access. (This directly aligns with your "Logical Deactivation" example).
- **GetEmployeeGovernmentIDs(employeeId):** Retrieves all statutory identification numbers associated with an employee (SSS, PhilHealth, TIN, Pag-IBIG) for compliance and reporting. Includes methods like `testGetSss()`, `testGetPhilHealth()`, `testGetTin()`, `testGetPagIbig()`.
- **GenerateEmployeeReports(criteria):** Compiles and formats comprehensive reports on employee data, allowing filtering by status, department, or other criteria. This leverages `testToString()` for formatted output.
- **GetMaxEmployeeID():** Retrieves the highest existing EID to ensure unique assignment for new employees.
- **ValidateEmployeeTableStructure():** Verifies the integrity and consistency of the database schema for the `Employees` table, ensuring all necessary columns and constraints are in place.
- **Data Impact:** Requires a dedicated `Employees` table storing all detailed employee attributes. A new `Status` or `isActive` column will be added to this table to support logical deactivation. Queries fetching active employees will need to include a filter (e.g., `WHERE Status = 'Active'`).

■ Allowance Management

- **Purpose:** To systematically define, track, and calculate various forms of supplementary compensation provided to employees, ensuring these amounts are accurately integrated into gross pay calculations.
 - **Description:** This module manages the different allowance types an employee is entitled to, allowing for the setting and retrieval of individual allowance amounts and their aggregation into a total allowance sum for payroll processing.
 - **Functionalities:**
 - **SetRiceAllowance(employeeId, amount):** Assigns or updates the specific monetary value or equivalent of the rice allowance for an employee.
 - **SetPhoneAllowance(employeeId, amount):** Assigns or updates the specific monetary value for an employee's phone allowance.
 - **SetClothingAllowance(employeeId, amount):** Assigns or updates the specific monetary value for an employee's clothing allowance.
 - **GetAllowanceDetails(employeeId):** Retrieves individual allowance amounts (Rice, Phone, Clothing) for a specified employee. This leverages methods like `testGetRiceAllowance()`, `testGetPhoneAllowance()`, `testGetClothingAllowance()`.
 - **GetHalfMonthRate(employeeId):** Retrieves the calculated half-month rate for an employee, primarily used for semi-monthly payroll calculations.
 - **GetHourlyRate(employeeId):** Retrieves the standard hourly rate for an employee, foundational for overtime and other time-based calculations.
 - **CalculateTotalAllowances(employeeId):** Computes and returns the sum of all applicable allowances for a given employee.
 - **ParseNumericAllowanceValues(valueString):** Converts string inputs for allowance amounts into a numeric format suitable for calculations, handling potential formatting issues.
 - **Data Impact:** Requires an `Allowances` table or allowance columns within the `Employees` table, linking specific allowance amounts to `EIDs`. Changes to allowance values will update these columns.
-

■ Deduction Management

- **Purpose:** To ensure the accurate calculation and application of mandatory statutory contributions and other authorized deductions from employee salaries, guaranteeing compliance with legal requirements and precise net pay determination.
- **Description:** This module handles all required deductions from an employee's gross pay, including government-mandated contributions (Pag-IBIG, PhilHealth, SSS) and withholding tax, to arrive at the correct net salary.
- **Functionalities:**
 - **SetPagIbigContribution(employeeId, amount):** Calculates and stores the employee's Pag-IBIG contribution based on their basic salary and current regulations.
 - **SetPhilHealthContribution(employeeId, amount):** Calculates and stores the employee's PhilHealth contribution based on their basic salary and prevailing health insurance contribution tables.
 - **SetSssContribution(employeeId, amount):** Calculates and stores the employee's SSS contribution according to their basic salary and the official SSS contribution schedule.
 - **SetWithholdingTax(employeeId, amount):** Calculates and records the income tax to be withheld from the employee's salary based on their taxable income and tax bracket.
 - **GetDeductionDetails(employeeId):** Retrieves individual deduction amounts (Pag-IBIG, PhilHealth, SSS, Withholding Tax) for a specific employee. This leverages `testGetPagIbigContribution()`, `testGetPhilHealthContribution()`, `testGetSssContribution()`, `testGetWithholdingTax()`.
 - **CalculateTotalDeductions(employeeId):** Computes and returns the aggregate sum of all calculated deductions for an employee for a given payroll period.
- **Data Impact:** Requires a `Deductions` table or deduction-related columns within the `Employees` table, linking calculated deduction amounts to `EIDs`. These values will be updated during each payroll cycle.

■ Incentive Management

- **Purpose:** To transparently track, calculate, and apply various forms of variable compensation such as overtime pay, performance-based bonuses, and holiday pay, ensuring all additional earnings are accurately accounted for and integrated into gross pay.
 - **Description:** This module manages the different types of incentives employees can earn beyond their regular salary, facilitating their computation and contribution to the employee's total gross income.
 - **Functionalities:**
 - **SetOvertime(employeeId, hours):** Records and stores the number of overtime hours worked by an employee.
 - **CalculateOvertimePay(employeeId):** Computes the monetary value of overtime work based on recorded hours and the employee's hourly rate, applying overtime multipliers.
 - **SetPerformanceBonus(employeeId, amount):** Assigns or updates the value of a performance bonus awarded to an employee.
 - **SetHolidayPay(employeeId, amount):** Calculates and records the special pay due for work performed on public holidays.
 - **GetIncentiveDetails(employeeId):** Retrieves individual incentive amounts (Overtime Pay, Overtime hours, Performance Bonus, Holiday Pay) for a specific employee. This leverages `testGetOverTimePay()`, `testGetOverTime()`, `testGetPerformanceBonus()`, `testGetHolidayPay()`.
 - **CalculateTotalIncentives(employeeId):** Computes and returns the aggregate sum of all incentives earned by an employee for a given period.
 - **Data Impact:** Requires an `Incentives` table or incentive-related columns within the `Employees` table, linking calculated incentive amounts to `EIDs`. These values will be updated based on performance or recorded overtime/holiday work.
-

■ Salary and Payroll Calculation

- **Purpose:** To act as the core processing engine for all compensation components, precisely calculating gross pay, net pay, and all contributing elements (allowances, deductions, incentives) in strict adherence to labor laws and company policies.

- **Description:** This module orchestrates the complex calculations involved in determining an employee's final take-home pay, ensuring accuracy, transparency, and compliance with financial regulations.
 - **Functionalities:**
 - **CalculateGrossPay(employeeId):** Computes an employee's total earnings before any deductions, by summing their basic salary, total allowances, and total incentives.
 - **CalculateNetPay(employeeId):** Determines the final take-home pay for an employee by subtracting their total deductions from their calculated gross pay.
 - **PerformAllCalculations(employeeId):** Executes a comprehensive sequence of calculations for a specified employee, including allowances, deductions, incentives, gross pay, and net pay, ensuring all dependent values are refreshed.
 - **GetFormattedDeduction(employeeId, deductionType):** Retrieves a specific deduction amount formatted for display (e.g., with currency symbols, decimal places).
 - **GetTotalAllowances(employeeId):** Retrieves the previously calculated total allowances for an employee.
 - **GetGrossPay(employeeId):** Retrieves the previously calculated gross pay for an employee.
 - **GetNetPay(employeeId):** Retrieves the previously calculated net pay for an employee.
 - **GetOvertimePay(employeeId):** Retrieves the previously calculated overtime pay for an employee.
 - **GetHolidayPay(employeeId):** Retrieves the previously calculated holiday pay for an employee.
 - **CheckCalculationOrderIndependence():** Verifies that the final payroll figures (e.g., net pay) remain consistent regardless of the sequence in which individual allowance, deduction, or incentive calculations are performed, ensuring robust financial logic.
 - **Data Impact:** Calculated payroll figures (Gross Pay, Net Pay, etc.) will be stored in a **PayrollRecords** table, linked to the **EID** and a specific payroll period. This table will serve as the historical record of all processed salaries.
-

■ Time Tracking and Work Hours Management

- **Purpose:** To provide a reliable system for employees to log their work hours, facilitating accurate attendance monitoring, calculation of hourly wages and overtime, and supporting leave accrual.
- **Description:** This module enables employees to record their clock-in and clock-out times, calculates total work durations, and processes this data for monthly hour summaries and timesheet generation.
- **Functionalities:**
 - **AddWorkTime(employeeId, clockInTime, clockOutTime):**
Records a new time log entry for an employee, capturing their start and end times for a specific work period.
 - **CalculateMinutesWorked(employeeId, timeLogId):** Computes the exact duration in minutes for a specific work log entry.
 - **CalculateMonthlyWorkHours(employeeId, month, year):**
Aggregates all recorded work times for an employee within a given month and year to determine their total monthly work hours.
 - **StartClock(employeeId):** Initiates a new work session for an employee, recording the clock-in timestamp.
 - **StopClock(employeeId):** Ends an active work session for an employee, recording the clock-out timestamp and triggering duration calculation.
 - **UpdateTimeLog(timeLogId, updatedData):** Allows authorized users to correct or modify existing time log entries (e.g., adjusting clock-in/out times due to forgotten punches).
 - **DeleteTimeLog(timeLogId):** Removes a specific time log entry from the system.
 - **GetTimesheetByEID(employeeId, startDate, endDate):**
Retrieves a comprehensive list of all time log entries for a specified employee within a defined date range, forming their timesheet.
- **Data Impact:** Requires a **TimeLogs** table storing **EID**, **ClockInTime**, **ClockOutTime**, and **DurationInMinutes** for each entry.

■ Leave Management

- **Purpose:** To streamline the entire employee leave process, from request submission and approval workflows to accurate tracking and historical record-keeping, ensuring adherence to company leave policies and labor laws.
- **Description:** This module provides a structured system for employees to submit leave requests (e.g., vacation, sick, personal), and for administrators/supervisors to review, approve, deny, or track these requests, maintaining a transparent audit trail.
- **Functionalities:**
 - **AddLeave(employeeId, leaveType, dateFiled, dateFrom, dateTo, reason):** Allows an employee to submit a new leave request, detailing the type of leave, filing date, requested date range, and a clear reason for absence. A unique **Leave ID** is generated.
 - **UpdateLeaveStatus(leaveId, newStatus, notes):** Enables authorized personnel to change the status of a leave request (e.g., "Pending", "Approved", "Rejected", "Cancelled") and add any relevant notes.
 - **GetLeaveById(leaveId):** Retrieves all details of a specific leave request using its unique identifier.
 - **GetEmployeeLeaves(employeeId):** Fetches a list of all leave requests (past and present) associated with a particular employee.
 - **GetAllLeavesTableModel():** Prepares and returns a comprehensive list of all leave requests in the system, typically formatted for display in a table, allowing administrators to review all pending and approved leaves.
 - **SortLeavesTable(column, order):** Arranges the displayed leave data in a table based on selected columns (e.g., **Date Filed**, **Employee Name**, **Status**) and sorting order (ascending/descending).
 - **SearchLeavesByEID(employeeId):** Filters and retrieves leave requests specifically for a given employee ID.
 - **DeleteLeave(leaveId):** Removes a specific leave record from the system (often a soft delete in practice).
- **Data Impact:** Requires a **Leaves** table storing **Leave ID**, **EID**, **LeaveType**, **DateFiled**, **DateFrom**, **DateTo**, **ReasonForLeave**, **Status**, and **Notes**.

■ Dashboard and Data Display

- **Purpose:** To provide a customizable, real-time overview of key employee and payroll metrics, enabling administrators and managers to quickly grasp operational status, identify trends, and make informed decisions.
 - **Description:** This module is responsible for aggregating, processing, and presenting various data points from different parts of the system onto a centralized dashboard or user interface, offering quick insights into the workforce and financial health.
 - **Functionalities:**
 - **ProcessEmployeeData():** Prepares employee-related data (e.g., total active employees, employees on leave) for dashboard display, often involving aggregations or counts.
 - **ProcessAllowanceData():** Aggregates and formats allowance-related data (e.g., total allowances disbursed last period) for display.
 - **ProcessDeductionData():** Aggregates and formats deduction-related data (e.g., total SSS contributions this month) for display.
 - **ProcessSalaryData():** Aggregates and formats salary-related data (e.g., total gross payroll, total net payroll) for dashboard display.
 - **GetDisplayValue(metricName):** Retrieves a specific calculated or processed metric value formatted for presentation on the dashboard.
 - **ClearDisplayData():** Resets or clears the current data displayed on the dashboard, preparing it for new data loading or a refresh.
 - **GetAllDisplayData():** Collects and returns all processed data points necessary to populate the entire dashboard view.
 - **Data Impact:** Primarily involves querying and processing data from **Employees**, **PayrollRecords**, **Allowances**, and **Deductions** tables. No direct write operations from this module, only data consumption and transformation for display.
-

■ Date and Time Utilities

- **Purpose:** To provide a consistent and accurate set of functions for handling all date and time operations within the system, ensuring data integrity, correct formatting for display, and reliable calculations based on time.
 - **Description:** This module offers helper functions for calculating date boundaries (e.g., first/last day of month), retrieving current timestamps, and converting date formats, crucial for accurate time tracking, payroll periods, and reporting.
 - **Functionalities:**
 - **GetFirstDayOfMonth(date):** Returns the date representing the first day of the month for a given input date.
 - **GetLastDayOfMonth(date):** Returns the date representing the last day of the month for a given input date.
 - **GetCurrentDate():** Retrieves the current system date.
 - **GetCurrentDateFormatted(format):** Retrieves the current system date formatted according to a specified string format (e.g., "YYYY-MM-DD", "MM/DD/YYYY").
 - **GetCurrentTimeFormatted(format):** Retrieves the current system time formatted according to a specified string format (e.g., "HH:MM:SS", "hh:mm A").
 - **ConvertStringToSqlDate(dateString):** Converts a date represented as a string (assuming valid formats) into a `java.sql.Date` object or equivalent, suitable for direct database insertion. This includes handling multiple valid input formats.
 - **Data Impact:** This is primarily a utility module and does not directly impact persistent data. It helps in formatting and preparing date/time values before they are stored or used in queries.
-

■ Database Query Management

- **Purpose:** To centralize, standardize, and secure the generation of database interaction queries, promoting code reusability, reducing errors, and bolstering security against common vulnerabilities like SQL injection.
- **Description:** This module provides a set of methods for constructing various types of SQL queries (search, sort, specific entity queries) in a consistent and often parameterized manner, abstracting the complexities of direct SQL string concatenation.

- **Functionalities:**
 - **CreateSearchQuery(columnName, searchValue):** Generates an SQL **SELECT** statement with a **WHERE** clause for searching a specific column in a default table.
 - **CreateSearchQuery(tableName, columnName, searchValue):** Generates an SQL **SELECT** statement with a **WHERE** clause for searching a specific column within a specified table.
 - **CreateSearchTimeLog(employeeId, startDate, endDate):** Constructs a specialized query to retrieve time logs for a given employee within a date range.
 - **CreateSortQuery(tableName, orderByColumn, order):** Generates an SQL **ORDER BY** clause to sort results from a specified table in ascending or descending order.
 - **CreateSortTimeLog(orderByColumn, order):** Generates an SQL **ORDER BY** clause specifically for time log data.
 - **CreateSortLeaveQuery(orderByColumn, order):** Generates an SQL **ORDER BY** clause specifically for leave records.
 - **GetQuery(queryKey):** Retrieves pre-defined or parameterized SQL query strings based on a unique identifier or key, ensuring consistency across the application.
 - **TestGettersAndSetters():** Verifies the correct functioning of internal getter/setter methods for query parameters or components.
 - **TestConstantQueryGetters():** Confirms that predefined, constant SQL query strings are retrieved correctly without modification.
- **Data Impact:** This module does not directly impact data; instead, it generates the SQL strings that other modules use to interact with the database. Proper implementation ensures data integrity during **SELECT**, **INSERT**, **UPDATE**, and **DELETE** operations.

■ User Authentication and Session Management

- **Purpose:** To establish a secure gateway to the system, verifying user identities, managing active user sessions, and enforcing role-based access control to ensure that only authorized personnel can access specific functionalities and sensitive data.

- **Description:** This module handles the complete login and session lifecycle, including credential validation, redirecting users based on their assigned roles, and maintaining the state of the logged-in user throughout their interaction with the application.
- **Functionalities:**
 - **AuthenticateUser(username, password):** Verifies the provided username and password against stored credentials. If valid, it returns the user's role and populates session data.
 - **AuthenticateAndRedirect(username, password):** Performs authentication and, upon success, automatically navigates the user to their designated dashboard or starting page based on their security role (e.g., admin dashboard, employee portal).
 - **SetLoggedInUser(userObject):** Stores the authenticated user's object or key information in the active session.
 - **GetLoggedInUser():** Retrieves the information of the currently logged-in user from the session.
 - **SetUsername(username):** Stores the username in the session.
 - **GetUsername():** Retrieves the username from the session.
 - **ClearSession():** Invalidates and clears all data associated with the current user session, effectively logging the user out.
 - **TestSingletonInstance():** Verifies that only a single instance of the `UserSessionSingleton` class can exist at any given time, ensuring consistent session state management.
 - **TestInitialState():** Confirms that a new session starts with a defined, empty, or default state.
 - **TestSetNullValues():** Verifies that session attributes can be correctly set to null, allowing for proper session state resets.
 - **TestMultipleUserSessions():** (Conceptual) Ensures the system can correctly manage distinct sessions for multiple concurrent users without inter-session data leakage.
- **Data Impact:** Stores temporary session data, typically in memory or a server-side session store, linked to a session ID. Does not directly impact persistent database records, but relies on `Employees` or `Users` tables for authentication credentials and roles.

■ One-Time Password (OTP) System

- **Purpose:** To provide an additional layer of security for critical transactions or account recovery, mitigating risks associated with compromised static passwords by requiring a temporary, unique, and time-sensitive verification code.
 - **Description:** This module generates unique, short-lived numeric or alphanumeric codes for authentication purposes and validates user-entered OTPs, enhancing the security posture of sensitive operations.
 - **Functionalities:**
 - **GenerateOTP(userId):** Creates a random, unique, and time-bound One-Time Password (OTP) for a specified user, typically storing it temporarily for verification.
 - **VerifyOTP(userId, otpCode):** Checks if the provided OTP matches the generated one for the user and if it is still within its validity period.
 - **Data Impact:** Requires a temporary storage mechanism (e.g., a cache, a dedicated **OTPs** table with an expiration timestamp) to store generated OTPs until they are verified or expire.
-

■ Restricted Input Handling

- **Purpose:** To ensure data quality and enhance system security by strictly controlling and validating user inputs, preventing the entry of malicious code, incorrect data types, or exceeding defined length limits.
- **Description:** This module implements input filters and validation rules that restrict characters, enforce data types (e.g., numbers only), and manage maximum lengths for various input fields across the application.
- **Functionalities:**
 - **InsertString(inputString):** Processes a string input, applying general filters to remove disallowed characters or ensure adherence to a pattern.
 - **InsertStringWithMaxLength(inputString, maxLength):** Processes a string input and truncates it if it exceeds a specified maximum length, preventing buffer overflows or excessive data storage.
 - **ReplaceCharacters(inputString, charToReplace, replacementChar):** Replaces specific characters within an input

string, often used for sanitization (e.g., converting ' to " for SQL safety).

- **NumbersOnly(inputString):** Filters a string to ensure it contains only numeric characters, rejecting or removing any letters or special symbols.
 - **SetAllowLetters(boolean):** Configures the input filter to permit or restrict alphabetic characters.
 - **SetAllowSpaces(boolean):** Configures the input filter to permit or restrict space characters.
 - **SetAllowHyphens(boolean):** Configures the input filter to permit or restrict hyphen characters.
 - **SetAllowApostrophes(boolean):** Configures the input filter to permit or restrict apostrophe characters (crucial for preventing SQL injection).
 - **SetAllowNumbers(boolean):** Configures the input filter to permit or restrict numeric characters.
 - **SetMaxLength(length):** Sets the maximum allowable length for string inputs, preventing excessive data entry.
 - **Data Impact:** Indirectly impacts data by ensuring that only clean, valid, and properly formatted data enters the system and is stored in the database. No direct database operations from this module.
-

■ Payslip Generation

- **Purpose:** To provide employees with a comprehensive, transparent, and easily understandable breakdown of their earnings, deductions, and net pay for a specific payroll period, ensuring compliance with legal requirements and fostering trust.
- **Description:** This module compiles all calculated payroll components—including basic salary, allowances, incentives, and deductions—into a structured, readable document (payslip or salary statement). It presents a detailed accounting of how an employee's net pay is derived, often supporting various output formats like print or digital (e.g., PDF).
- **Functionalities:**
 - **GeneratePayslip(employeeId, payrollPeriodId):** Initiates the process of compiling and rendering a payslip for a specific

employee and a designated payroll period. This function acts as an orchestrator, pulling data from various sources.

- **RetrievePayslipData(employeeId, payrollPeriodId):** Gathers all necessary financial data points, including:
 - Employee's personal details (name, EID, designation) from **Employee Data Management**.
 - Basic Salary, Gross Semi-monthly Salary, Hourly Rate from **Employee Data Management**.
 - All individual and total allowance amounts from **Allowance Management**.
 - All individual and total deduction amounts from **Deduction Management**.
 - All individual and total incentive amounts from **Incentive Management**.
 - Calculated Gross Pay and Net Pay from **Salary and Payroll Calculation**.
 - Work hours data (if applicable for hourly employees) from **Time Tracking and Work Hours Management**.
- **FormatPayslipContent(payslipData):** Structures the retrieved data into a predefined payslip template, ensuring clear labeling, correct formatting of monetary values (e.g., currency symbols, decimal places), and logical grouping of earnings and deductions.
- **ExportPayslip(payslipContent, formatType):** Provides options to output the formatted payslip. Common format types include:
 - **Printable Format:** Renders the payslip suitable for physical printing.
 - **PDF Document:** Generates a secure, non-editable PDF file of the payslip for digital distribution or archiving.
- **ArchivePayslip(employeeId, payrollPeriodId, payslipFileLocation):** (Optional but highly recommended) Stores a copy of the generated payslip document in a secure, accessible location, linked to the employee and payroll period, for historical records and easy retrieval.
- **Data Impact:**
 - **Reads From:** **Employees** table (for personal and basic salary details), **Allowances** table, **Deductions** table, **Incentives** table,

`PayrollRecords` table (for gross/net pay results), and potentially `TimeLogs` (for detailed work hours).

- **Writes To:** Potentially an `ArchivedPayslips` directory or a `PayslipHistory` table, storing the generated document or a link to its location, along with metadata (employee ID, period, generation date). It does *not* modify the core financial calculation tables.

3.3. Non-Functional Requirements

The expanded MotorPH payroll system will be developed with a strong emphasis on key **non-functional requirements** to ensure its reliability, performance, and security. It will be designed for **high availability and reliability**, minimizing downtime and ensuring continuous payroll operations. The system will prioritize **performance and responsiveness**, handling large volumes of transactions and data retrieval efficiently. **Robust security measures** will protect sensitive employee and payroll data through access controls, encryption, and audit capabilities. Furthermore, the system will emphasize **usability** for an intuitive user experience, **scalability** to accommodate future growth, and **maintainability** for easy updates and troubleshooting, ensuring a stable, efficient, and adaptable payroll solution.

■ Security

- **Purpose:** To safeguard the confidentiality, integrity, and availability of all sensitive employee and payroll data, as well as system resources, from unauthorized access, modification, or disclosure, thereby ensuring compliance with data protection regulations and fostering user trust.
- **Description:** The system must implement robust measures to protect against various cyber threats, including unauthorized logins, data breaches, and malicious injections. This involves secure authentication, input validation, and secure handling of sensitive information.
- **Specifics from tests (Detailed):**
 - **Invalid Login Handling (`testInvalidLogin_IncorrectPassword()`):**
The system must prevent unauthorized access by:

- Not differentiating between "incorrect username" and "incorrect password" messages to thwart enumeration attacks.
- Implementing account lockout policies after a configurable number of failed login attempts.
- Logging all failed login attempts for auditing and anomaly detection. This ensures that attackers cannot easily guess credentials through brute force.
- **OTP System Integrity (`testGenerateOTP()`, `testVerifyOTP()`):**
The OTP mechanism must be resistant to various attacks:
 - **Randomness:** Generated OTPs must be truly random and unpredictable to prevent pre-calculation.
 - **Time-limited Validity:** OTPs must expire after a very short, configurable duration (e.g., 5 minutes) to prevent replay attacks.
 - **Single Use:** Each OTP must be invalidated immediately after its first successful verification.
 - **Rate Limiting:** Implement limits on the number of OTP generation requests and verification attempts to prevent abuse.
- **Data Input Sanitization (`testInsertString_InvalidInput()`, `testReplace_InvalidInput()`):** The system must aggressively filter and sanitize all user-supplied inputs to neutralize malicious content. This includes:
 - **SQL Injection Prevention:** Using parameterized queries (prepared statements) for all database interactions and strictly escaping or rejecting potentially harmful characters (like single quotes) in user input.
 - **Cross-Site Scripting (XSS) Prevention:** Encoding or sanitizing any user-generated content before it is displayed to prevent injection of client-side scripts.
 - **Buffer Overflow Prevention:** Enforcing strict maximum length limits on input fields to prevent attackers from sending oversized data that could crash the application or overwrite memory.

■ Reliability and Robustness

- **Purpose:** To guarantee that the system performs its intended functions consistently and accurately under all anticipated conditions, including erroneous inputs, high data volumes, or unexpected scenarios, without failing, losing data, or producing incorrect outputs. This ensures system stability and user confidence.
- **Description:** The system is designed to be resilient, capable of handling errors gracefully, maintaining data integrity even under stress, and ensuring that core calculations yield precise results regardless of operational sequences.
- **Specifics from tests (Detailed):**
 - **Data Consistency (`testDataConsistency()` for `EmployeeDetails`):**

The system must rigorously enforce internal consistency rules across all related data. For instance, if an employee's basic salary is updated, all dependent calculations (e.g., allowances, deductions, gross pay) must automatically reflect this change to ensure financial records always reconcile. This prevents discrepancies that could lead to incorrect payroll or reporting.
 - **Error Handling for Invalid Data (`testConvertStringToSqlDate_InvalidFormat()`, `testAddTimeLog_NullData()`, `testUpdateTimeLog_InvalidData()`):** The system must not crash or produce unexpected behavior when presented with malformed, incomplete, or out-of-bounds data. Instead, it should:
 - Validate inputs at the point of entry and provide clear, actionable error messages to the user.
 - Log validation failures for debugging and monitoring.
 - Prevent invalid data from being stored in the database, maintaining data quality.
 - **Boundary Value Handling (`testSetZeroBasicSalary()`, `testSetVeryLargeBasicSalary()`):** The system's logic and calculations must be verified to correctly handle extreme valid input values. This includes zero values (e.g., zero basic salary for specific employment types) and very large values (e.g., maximum

allowable salary), ensuring that the system does not overflow, underflow, or miscalculate in edge cases.

- **Graceful Handling of Non-existent Records**

(**testUpdateLeaveStatus_NotFound()**, **testDeleteLeave_NotFound()**): When an operation is attempted on a record that legitimately does not exist in the database (e.g., updating a leave request with an invalid ID), the system must:

- Not throw a system error or crash.
- Provide a clear, user-friendly message indicating that the record was not found, allowing the user to understand and correct their input.
- Prevent any unintended side effects or phantom updates.

- **Calculation Order Independence**

(**testCalculationOrderIndependence()**): For complex financial calculations involving multiple steps (like payroll, which combines allowances, deductions, and incentives), the final result must be deterministic. This means the outcome should be identical regardless of the sequence in which the sub-calculations are performed, ensuring the mathematical model is robust and reliable.

■ Data Integrity

- **Purpose:** To guarantee that all data stored and processed by the system is accurate, consistent, valid, and complete at all times, preventing corruption, unauthorized alteration, or unintended loss.
- **Description:** The system employs various mechanisms, including schema validation, input validation, and controlled query generation, to ensure the fidelity and reliability of its stored information.
- **Specifics from tests (Detailed):**
 - **Employee Table Structure Validation**
(**testEmployeeTableStructureUsingQuery()**): The system must perform checks, ideally at deployment or during integrity scans, to confirm that the database tables (e.g., **Employees**, **TimeLogs**, **Leaves**) adhere to their defined schema. This includes verifying correct data types (e.g., **INT** for EID, **VARCHAR** for names, **DATE** for birthdays), nullability constraints (e.g., **NOT NULL** for essential

fields), and primary/foreign key relationships. This prevents structural inconsistencies that could lead to data corruption or application errors.

- **Input Restrictions** (**testNumbersOnly()**, **testSetAllowLetters()**, **testSetMaxLength()**): A proactive approach to data integrity involves validating data at the point of entry. This means:
 - Enforcing data type constraints (e.g., ensuring a "salary" field only accepts numeric input).
 - Restricting character sets (e.g., allowing only letters and spaces for names, or disallowing special characters in usernames).
 - Limiting input lengths to prevent oversized data from causing database truncation or schema violations. This prevents bad data from ever entering the system.
- **Query String Correctness** (**testQueryStringCorrectness()**): All SQL queries constructed and executed by the application must be syntactically perfect and logically sound. This ensures that database operations (**SELECT**, **INSERT**, **UPDATE**, **DELETE**) perform exactly as intended, preventing unintended data modifications, deletions, or retrievals. This is closely tied to security, as correctly formed queries (especially parameterized ones) also prevent SQL injection attacks.

■ Usability

- **Purpose:** To ensure the system is intuitive, efficient, and user-friendly for all target users (employees, HR staff, administrators), minimizing the learning curve, reducing errors, and maximizing productivity through clear interfaces and understandable information presentation.
- **Description:** The system aims to provide a clear and straightforward user experience by presenting information in easily digestible formats and offering intuitive controls for common tasks.
- **Specifics from tests (Detailed):**
 - **Clear Display** (**testClearDisplayData()**): The presence of a function to clear or reset displayed data implies an emphasis on maintaining a clutter-free and manageable user interface. This allows users to quickly refresh information, prepare for a new

input session, or clear old search results, contributing to a fluid workflow.

- **Formatted Output** (**testDateFormatAccuracy()**, **testGetCurrentDateFormatted()**, **testGetCurrentTimeFormatted()**): Dates, times, and financial figures presented to the user must be in easily readable, consistent, and contextually appropriate formats. This avoids ambiguity (e.g., distinguishing MM/DD/YY from DD/MM/YY), improves comprehension, and enhances the overall professional appearance and user experience of reports and dashboards.

■ Performance

- **Purpose:** To ensure the system delivers a responsive and efficient user experience by completing operations, calculations, and data retrievals within acceptable timeframes, even under typical or peak load conditions.
- **Description:** The system is engineered to execute complex calculations and data processing tasks rapidly, minimizing latency and ensuring that users do not experience noticeable delays during critical interactions.
- **Specifics from tests (Detailed):**
 - **Efficient Calculations** (**testPerformAllCalculations()** for **SalaryCalculator**): The performance of the comprehensive salary calculation routine is critical. The system is expected to process all payroll components (gross, net, allowances, deductions, incentives) for a single employee or potentially batches of employees quickly, ensuring that payroll runs are completed efficiently and users don't wait long for financial figures.
 - **Data Processing Efficiency** (**testProcessEmployeeData()**, **testProcessAllowanceData()** etc. for **DashboardData**): The system must be capable of rapidly aggregating, filtering, and preparing large volumes of raw data from various modules into digestible metrics suitable for dashboard display or reports. This directly impacts the responsiveness of dashboards and the timeliness of management insights.

- **Time Logging/Calculation (`testAddWorkTime()`, `testCalculateMinutesWorked()`, `testCalculateMonthlyWorkHours()`):** Daily operational tasks like recording clock-in/out times, calculating individual shift durations, or summarizing monthly work hours must be executed with minimal delay. This ensures a smooth experience for employees when punching in/out and for administrators when reviewing attendance records.
-

■ Maintainability

- **Purpose:** To ensure the system's codebase is structured in a way that makes it easy to understand, modify, debug, and extend with new features or adaptations to changing requirements, thereby minimizing the long-term cost and effort associated with software evolution.
- **Description:** The system adheres to principles of modularity, clear separation of concerns, and good coding practices to facilitate ongoing development, troubleshooting, and future scalability.
- **Specifics from tests (Detailed):**
 - **Modular Test Structure:** The comprehensive and organized list of tests, grouped by distinct functional areas (e.g., `Allowances`, `Deductions`, `EmployeeDetails`, `SalaryCalculator`), strongly indicates a modular software architecture. This design principle means that components are self-contained and loosely coupled, making it easier for developers to work on one part of the system without inadvertently affecting others, greatly simplifying maintenance and updates.
 - **Getter/Setter Tests (`testGetSet...()`, `testSet...()` methods):** The prevalence of tests specifically for getter and setter methods highlights adherence to the principle of encapsulation. This practice means that the internal state and data structures of a class are hidden and can only be accessed or modified through well-defined public methods. This design makes the code easier to understand, debug (as data changes are controlled), and refactor without breaking external dependencies, significantly enhancing long-term maintainability.

■ Scalability (Future Focused)

- **Purpose:** To ensure the system can efficiently handle a growing number of employees, increased data volumes, and expanding transaction loads without significant degradation in performance or requiring a complete architectural overhaul, thus supporting the organization's long-term growth.
- **Description:** The system is designed with an architecture that allows for incremental growth in terms of user concurrency, the number of employee records, and the volume of daily transactions (e.g., time logs, leave requests, payroll calculations). This involves anticipating future demands and ensuring the underlying infrastructure and software design can accommodate them, often through efficient resource utilization and modularity.
- **Specifics from tests (Inferred or Design Considerations):**
 - **Efficient Calculations (`testPerformAllCalculations()`, `testCalculateGrossPay()`):** While not explicitly testing scalability, the emphasis on efficient calculation implies that the system aims to process payroll quickly. For true scalability, these calculations would need to be optimized to handle thousands of employees without linear increases in processing time, perhaps through batch processing or optimized algorithms.
 - **Data Processing Efficiency (`testProcessEmployeeData()`, `testProcessAllowanceData()` for `DashboardData`):** The ability to quickly process and display large datasets for dashboards suggests an underlying design that prioritizes data retrieval and aggregation performance, which is a prerequisite for scalability. As data grows, these processes must remain performant.
 - **Database Query Optimization (Implied by `DBQueries` module):** The presence of a dedicated `DBQueries` module with methods for creating search and sort queries hints at a design where database interactions are managed. For scalability, these queries must be indexed, optimized, and potentially capable of supporting distributed database solutions as data volume increases.

- **Modular Design (Inferred from various distinct modules like Allowances, Deductions, SalaryCalculator):** The system's modularity, evident from the distinct test scenarios, supports scalability. Individual modules can potentially be optimized, upgraded, or even deployed independently (e.g., as microservices) as the system grows, preventing bottlenecks in one area from affecting the entire application.
 - **User Session Management (testMultipleUserSessions()):** While the test might confirm distinct sessions, true scalability would require the session management to efficiently support hundreds or thousands of concurrent users without exhausting server resources or introducing noticeable lag.
 -
-

■ Legal and Compliance

- **Purpose:** To ensure the system adheres strictly to all relevant national and local labor laws, taxation regulations, social security mandates, and data privacy acts (e.g., HIPAA, GDPR, or local equivalents like the Philippine Data Privacy Act of 2012), minimizing legal risks and avoiding penalties.
- **Description:** The system is built to incorporate and automatically apply the necessary rules and calculations dictated by legal frameworks, particularly concerning employee benefits, deductions, working hours, and data handling. It ensures that all generated reports and financial outputs are compliant and auditable.
- **Specifics from tests (Detailed):**
 - **Statutory Deduction Calculations (testSetPagIbigContribution(), testSetPhilHealthContribution(), testSetSssContribution()):** These tests directly verify the correct calculation and application of mandatory government contributions (Pag-IBIG, PhilHealth, SSS), which are strictly governed by national laws in the Philippines. This ensures the system meets its legal obligation for accurate deductions.
 - **Withholding Tax Calculation (testSetWithholdingTax()):** This test confirms the system's ability to correctly calculate and apply

income tax withholding based on current tax laws and employee tax statuses, a critical aspect of tax compliance.

- **Accurate Time Logging and Calculation (`testAddWorkTime()`, `testCalculateMinutesWorked()`, `testCalculateMonthlyWorkHours()`):** Accurate tracking of work hours is essential for compliance with labor laws regarding minimum wage, overtime pay eligibility, and maximum working hours. These tests ensure the foundational data for compliance is correct.
- **Payslip Generation (New Module `GeneratePayslip`):** The payslip is a legally required document in many jurisdictions. The system's ability to generate comprehensive, accurate, and properly formatted payslips ensures that employees receive the legally mandated breakdown of their earnings and deductions.
- **Logical Deactivation of Employee Records (from `Employee Data Management`):** Instead of physical deletion, logically deactivating records (as per your example) is crucial for compliance with data retention policies, auditing requirements, and potentially investigations, as it preserves historical employment data.
- **Data Input Restrictions (`testSetAllowApostrophes()`, `testInsertString_InvalidInput()`):** While primarily a security feature, preventing certain characters or invalid data input indirectly supports compliance by ensuring data integrity, which is a prerequisite for reliable reporting to regulatory bodies.
- **Data Privacy (Implicit in Security and Session Management):** While not explicitly named as a test, the presence of `OneTimePin` authentication and `UserSessionSingleton` for managing logged-in users strongly implies a commitment to data privacy. To fully comply with regulations like the Data Privacy Act of 2012 (Philippines), the system would also need features for consent management, data access logging, secure data storage (encryption), and data deletion policies.

■ Portability

- **Purpose:** To ensure the system can be easily deployed, migrated, or run with minimal effort across different operating systems, hardware configurations, or database environments, providing deployment flexibility and reducing vendor lock-in and migration costs.
- **Description:** The system is designed to minimize its dependency on a specific software or hardware stack. This involves utilizing platform-agnostic technologies, abstracting underlying infrastructure (like databases), and avoiding highly specialized configurations, allowing it to function effectively in diverse deployment scenarios, including on-premise servers, virtual machines, or various cloud platforms.
- **Specifics from tests (Inferred from Design & Best Practices):**
 - **Database Abstraction (Inferred from DBQueries module):** The presence of a dedicated DBQueries module that generates and manages SQL queries suggests that the application logic does not directly embed database-specific SQL syntax. If implemented correctly (e.g., using a database abstraction layer or ORM behind DBQueries), this allows the system to switch between different database management systems (e.g., MySQL, PostgreSQL, SQL Server) with minimal code changes, a key aspect of portability.
 - **Modular Design (Evident from distinct components like Allowances, Deductions, SalaryCalculator):** A well-defined modular architecture, where functionalities are encapsulated in separate components, inherently supports portability. These modules can be more easily decoupled from specific platform features and reused or redeployed in different environments without affecting the entire system.
 - **Standardized Language and Frameworks (Assumed):** While not explicitly stated in the test cases, the use of widely adopted programming languages (like Java, Python, C#) and standard frameworks implies that the code itself is not tied to a proprietary ecosystem, allowing it to compile and run on various operating systems (e.g., Windows, Linux) without extensive modifications.
 - **External Dependency Management (Design Consideration):** To enhance portability, the system should ideally externalize configuration (e.g., database connection strings, API keys) rather than hardcoding them. This allows the application to adapt to

new environments by simply changing configuration files, rather than requiring code recompilation or major refactoring.

- **Date Format Agnosticism (Inferred from Date and Time Utilities):** The presence of `testConvertStringToSqlDate_ValidFormat1()` and `testConvertStringToSqlDate_ValidFormat2()` implies the system can handle different date input formats and convert them to a standardized internal/SQL format. This flexibility is crucial for portability across regions with different date conventions.

4. Appendix

4.1. Glossary

This glossary defines key terms, acronyms, and abbreviations used throughout this Software Requirements Specification (SRS) document. It aims to ensure a common understanding among all stakeholders, from technical developers to HR and payroll personnel. Definitions are tailored where necessary for project-specific context.

A

- **ACID Properties:** A set of properties (Atomicity, Consistency, Isolation, Durability) that guarantee that database transactions are processed reliably. Crucial for maintaining data integrity, especially in payroll systems.
- **Active Employee:** An employee whose record is currently marked as "active" in the system, indicating they are currently employed by MotorPH and included in active payroll runs and employee listings.
- **API (Application Programming Interface):** A set of defined rules that enable different applications to communicate with each other. (Mentioned as a future integration capability).
- **Audit Trail:** A chronological sequence of records, providing documentary evidence of the sequence of activities that have affected a specific

operation, procedure, or event. In this system, it tracks changes to employee data and payroll processing.

- **Authentication:** The process of verifying the identity of a user, system, or process (e.g., username and password).
 - **Authorization:** The process of determining what an authenticated user is permitted to do or access within the system.
-

B

- **BIR (Bureau of Internal Revenue):** The primary government agency in the Philippines responsible for the collection of internal revenue taxes. Payroll computations must comply with BIR regulations.
 - **Boilerplate Code:** Sections of code that are repeated in multiple places with little to no variation. Often associated with direct JDBC usage.
-

C

- **Codebase:** The entire collection of source code for a software system.
 - **Connection Pooling:** A technique used to improve the performance of applications by managing a pool of reusable database connections, reducing the overhead of opening and closing new connections for each request.
 - **CSV (Comma Separated Values):** A simple file format used to store tabular data, where each line represents a data record, and fields are separated by commas.
 - **Customized Query Optimization:** The process of specifically tuning SQL queries to achieve better performance for particular business rules or data retrieval needs, often done manually with JDBC.
-

D

- **Database Triggers:** Stored procedures in a database that automatically execute or "fire" when a specific event occurs (e.g., an insert, update, or delete operation) on a table. Used for logical deletion.

- **DBCP (Database Connection Pooling):** Refers to Apache Commons DBCP, a popular library for implementing connection pooling in Java applications.
 - **Deactivation (Logical Deletion):** The process of marking an employee record as "inactive" in the database by setting a status flag, rather than physically removing the record. This preserves historical data.
 - **Dependency Injection:** A software design pattern that allows the creation of dependent objects outside of a class and provides those objects to a class. Used to improve testability and modularity (future architectural goal).
 - **Desktop Application:** A software application that runs natively on a computer's operating system (e.g., Windows, macOS), as opposed to a web-based or mobile application.
-

E

- **Employee ID:** A unique identifier assigned to each employee within the MotorPH payroll system.
 - **Employee Lifecycle Management:** The process of managing an employee's journey within an organization, from hiring to onboarding, active employment, and eventual separation or re-hire.
 - **Encryption:** The process of converting information or data into a code, especially to prevent unauthorized access.
 - **ERP (Enterprise Resource Planning):** A type of software that organizations use to manage day-to-day business activities such as accounting, procurement, project management, risk management and compliance, and supply chain operations. (Mentioned as an exclusion for current integration).
 - **Export Capabilities:** The functionality to generate data or reports from the system in various external file formats (e.g., PDF, CSV, Excel).
-

H

- **HRIS (Human Resources Information System):** A system that manages HR data and processes, such as employee records, benefits

administration, and talent management. (Mentioned as an exclusion for current integration).

I

- **IDE (Integrated Development Environment):** A software application that provides comprehensive facilities to computer programmers for software development. NetBeans IDE 25 is the chosen IDE.
 - **Inactive Employee:** An employee whose record is marked as "inactive" in the system, typically due to resignation, termination, or retirement. Their historical data is retained.
 - **Indexing:** A database technique used to speed up data retrieval operations by creating special lookup tables.
 - **Integration:** The process of combining different software systems or modules to work together seamlessly.
 - **Internationalization (i18n):** The process of designing a software application so that it can be adapted to various languages and regions without engineering changes. (Mentioned as an exclusion).
-

J

- **Java Database Connectivity (JDBC):** A Java API that defines how a client can connect to a database. It provides methods for querying and updating data in a database.
 - **JDK (Java Development Kit):** A software development environment used for developing Java applications. JDK 25 is specified.
 - **JUnit:** A unit testing framework for the Java programming language.
-

L

- **Localization (l10n):** The process of adapting internationalized software for a specific region or language by adding locale-specific components and translating text. (Mentioned as an exclusion).
- **Logical Deletion:** See Deactivation.

M

- **Model (MVVM):** In the Model-View-ViewModel pattern, the Model represents the data and business logic of the application.
- **MotorPH:** The name of the company for which the payroll system is being developed/expanded.
- **MVVM (Model-View-ViewModel):** An architectural pattern used to separate the user interface logic from the business logic and data. Improves maintainability and testability.
- **MySQL Workbench 8.0:** A visual database design tool for MySQL databases.

N

- **NetBeans IDE 25:** The specific version of the Integrated Development Environment chosen for developing the system.
- **Non-Functional Requirements (NFRs):** Criteria that describe how a system performs a function, rather than the function itself (e.g., performance, security, usability, reliability).

O

- **ORM (Object-Relational Mapping):** A programming technique for converting data between incompatible type systems using object-oriented programming languages. (Not chosen for this project, but considered).
- **Optimization:** The process of modifying a system to make some aspect of it perform more efficiently (e.g., SQL query optimization).

P

- **Pag-IBIG (Home Development Mutual Fund):** A national savings program and affordable housing finance program in the Philippines. Employee contributions are mandatory.
- **Parameterized Queries:** SQL queries that use placeholders for values instead of hardcoding them, improving security (preventing SQL injection) and reusability.
- **Paystub:** A document given to an employee detailing the components of their pay, including gross pay, deductions, and net pay.
- **PDF (Portable Document Format):** A file format used to present documents in a manner independent of application software, hardware, and operating systems.
- **Performance Testing Environment:** A dedicated setup used to test the system's performance under various loads and conditions.
- **PhilHealth (Philippine Health Insurance Corporation):** A government-owned and controlled corporation in the Philippines that provides health insurance coverage and ensures affordable health care services for all citizens. Employee contributions are mandatory.
- **Physical Deletion:** The permanent removal of a record from the database. (Avoided in this expansion in favor of logical deletion).
- **Predictive Analytics:** The use of statistical algorithms and machine learning techniques to identify the likelihood of future outcomes based on historical data. (Mentioned as an exclusion for current phase).
- **Project Scope:** The defined boundaries of a project, outlining what is and what is not included.

R

- **RBAC (Role-Based Access Control):** A method of restricting system access based on the roles of individual users within an enterprise.
 - **Reactivation:** The process of changing a logically deactivated employee's status back to "active" within the system.
 - **RESTful APIs:** An architectural style for designing networked applications. (Mentioned as a future integration capability).
 - **Rollback:** The process of reverting a database or transaction to a previous state, typically to undo changes made due to an error.
-

S

- **Scalability:** The capability of a system, network, or process to handle a growing amount of work, or its potential to be enlarged to accommodate that growth.
- **Schema (Database Schema):** The logical configuration and structure of an entire database, including tables, fields, relationships, indexes, and constraints.
- **Security Compliance:** Adherence to established standards, policies, and regulations aimed at protecting data and systems from unauthorized access or breaches.
- **Service-Oriented Architecture (SOA):** An architectural style where discrete components (services) provide functionalities to other components via a communication protocol over a network. (Mentioned as a future architectural goal).
- **Single Sign-On (SSO):** An authentication process that allows a user to access multiple applications with one set of login credentials. (Mentioned as an exclusion for current integration).
- **SQL (Structured Query Language):** A standard language for storing, manipulating, and retrieving data in relational databases.
- **SRS (Software Requirements Specification):** A document that describes the intended purpose, functionalities, and constraints of a software system.
- **SSS (Social Security System):** A social insurance program in the Philippines that provides benefits to private sector employees and their families in times of contingencies such as sickness, maternity, disability, retirement, and death. Employee contributions are mandatory.
- **Stakeholders:** Individuals or groups who have an interest in or are affected by the project, including users, management, and developers.
- **Stored Procedures:** A subroutine available to applications that access a relational database system. It's stored in the database data dictionary. Used for routine tasks and complex logic.
- **System Constraints:** Limitations or boundaries on the system's design and implementation (e.g., technology choice, operating environment).

T

- **Testability:** The degree to which a software system or component can be tested.
 - **Transaction Management:** The process of managing database operations as atomic units, ensuring data integrity.
 - **UI (User Interface):** The point of interaction and communication between a human user and a computer program.
 - **Unit Testing:** A software testing method by which individual units of source code, sets of one or more computer program modules together with their associated control data, usage procedures, and operating procedures, are tested to determine whether they are fit for use.
-

V

- **Validation (Data Validation):** The process of ensuring that data is clean, correct, and useful.
- **View (Database View):** A virtual table based on the result-set of a SQL statement. Used to simplify queries and control data access.
- **View (MVVM):** In the MVVM pattern, the View represents the user interface elements that are visible to the user.
- **ViewModel (MVVM):** In the MVVM pattern, the ViewModel acts as an intermediary between the View and the Model, handling the presentation logic and exposing data to the View.

4.2. Diagrams

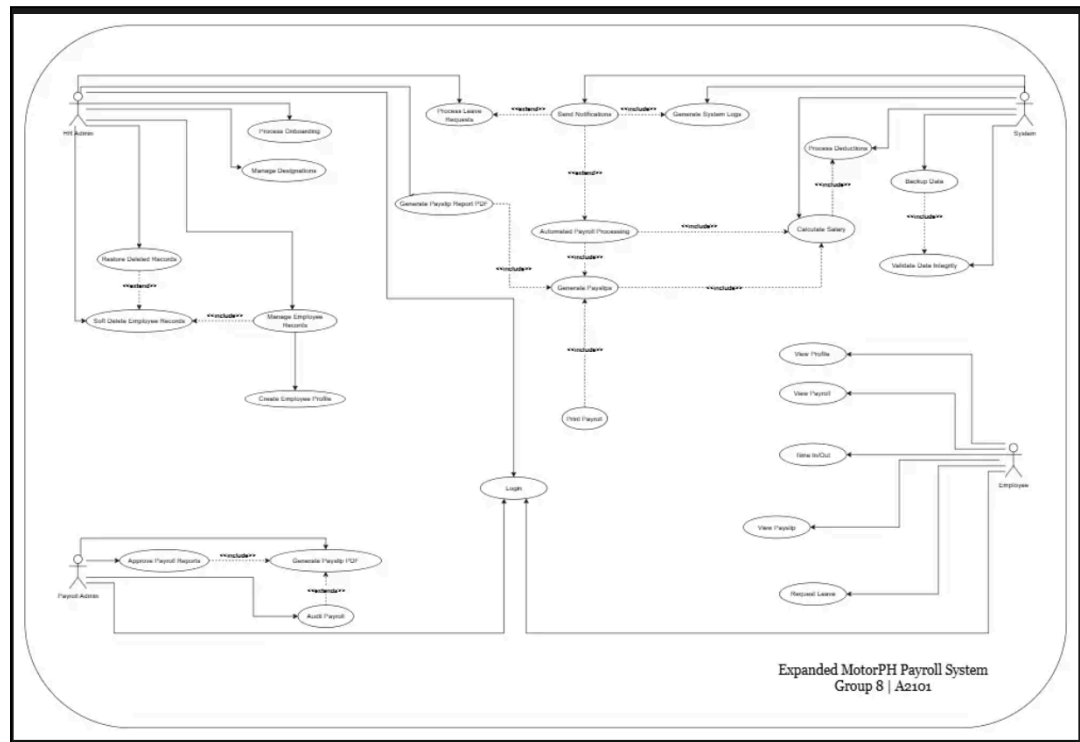


Fig. 1: Use Case Diagram for Expanded MotorPH Payroll System
(click link below to enlarge image)

■ **Link to Use Case Diagram:**

https://drive.google.com/file/d/16ilPmT8tFZ_vaeL-W4qV4IYy0j-JOJcD/view?usp=sharing

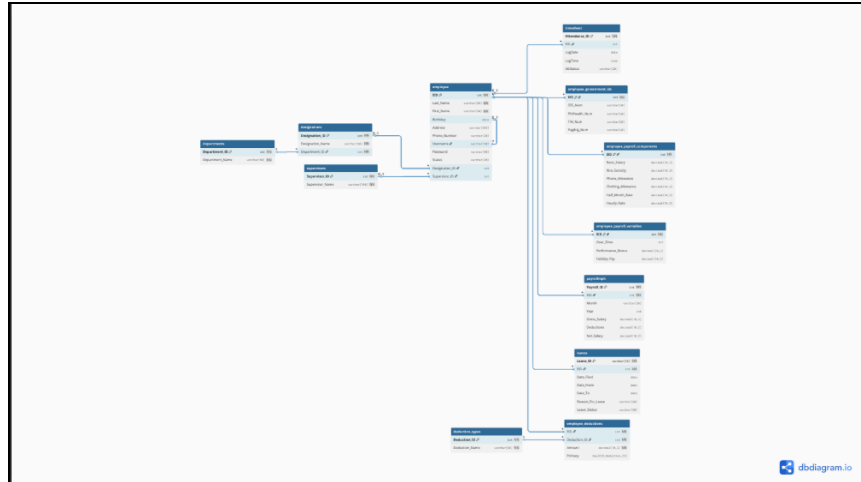


Fig. 2: Entity Relationship Diagram for Expanded MotorPH Payroll System
(click link below to enlarge image)

- **Link to Entity Relationship Diagram:**
https://drive.google.com/file/d/1yLA7PXKzlFLEAydzcMclFfjE9xYh9RL/view?usp=drive_link

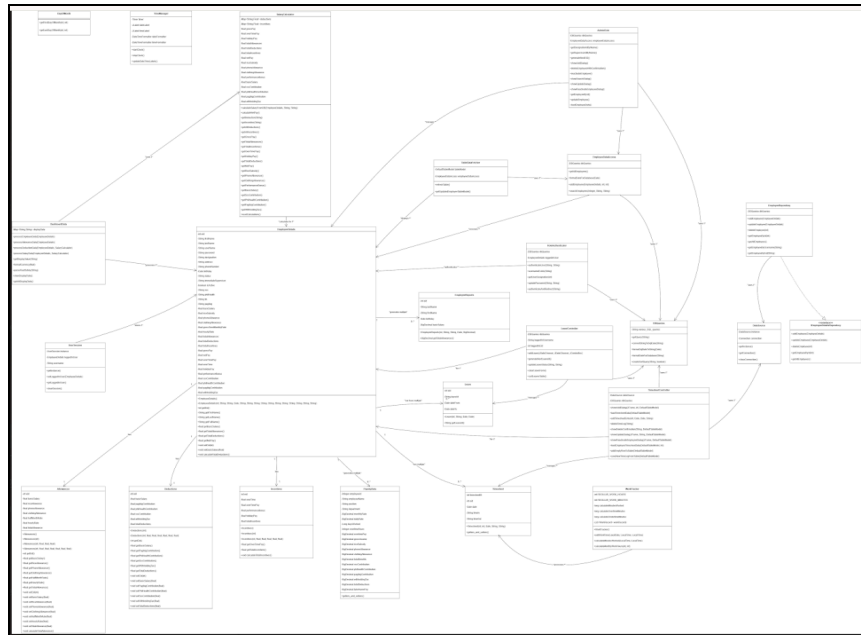


Fig. 3: Class Diagram for Expanded MotorPH Payroll System
(click link below to enlarge image)

- **Link to Class Diagram:**
https://drive.google.com/file/d/1_ZwKxunyK_G11g0LF7B_YkVWK_ZkxZDK/view?usp=sharing

